

Community BookHub Rental Tracker

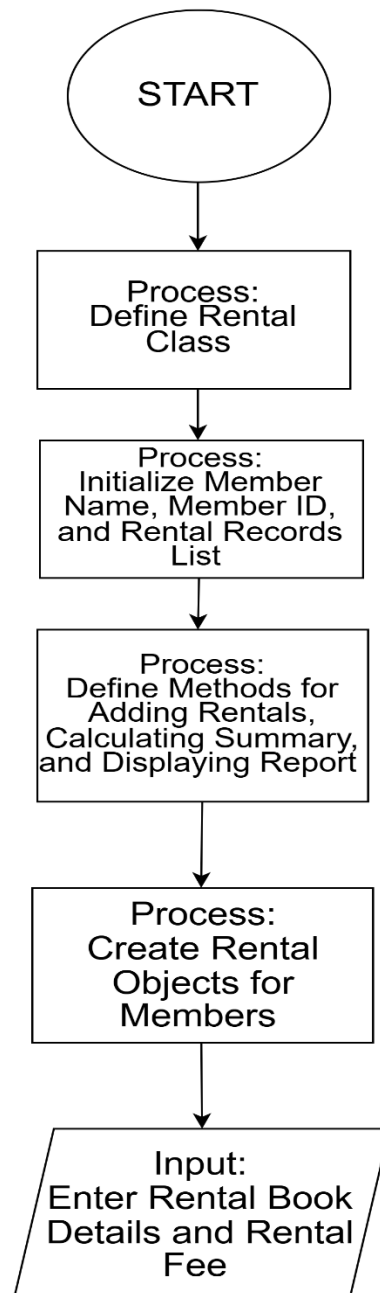
Part 1: Algorithm Design

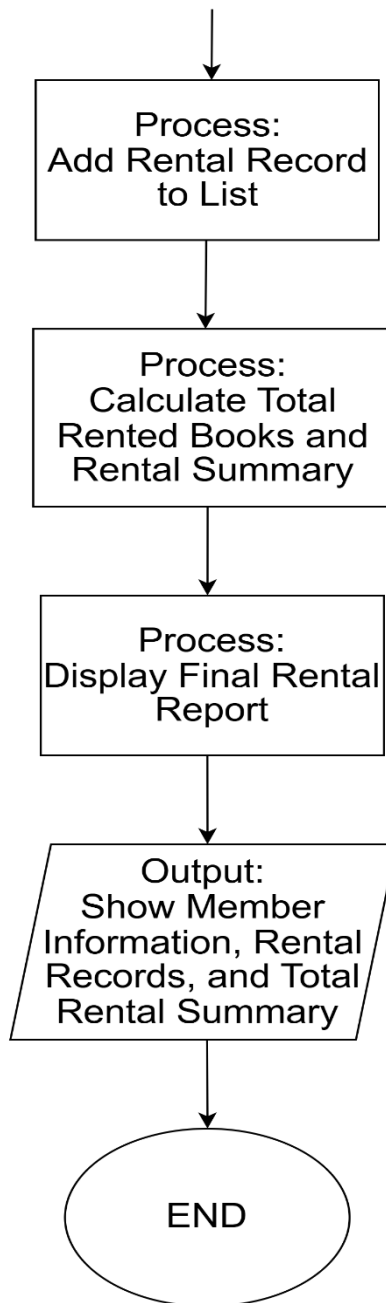
1. Start the program.
2. Define a base class called Rental as a blueprint.
3. Set up the attributes: member's name, member's ID, list of rental progress
4. Define a method to add rental books
5. Define a method to display the member's details.
6. Define a Method to calculate the rental summary
7. Define a method display the final report.
8. Create rental objects for different members.
9. Add rental records and display the final rental report.
10. End the program.

Part 2: Logic Flowchart

- [Start] (Oval)
- Process: Define Rental Class (Rectangle)
- Process: Initialize Member Name, Member ID, and Rental Records List (Rectangle)
- Process: Define Methods for Adding Rentals, Calculating Summary, and Displaying Report (Rectangle)
- Process: Create Rental Objects for Members (Rectangle)

- Input: Enter Rental Book Details and Rental Fee (Parallelogram)
- Process: Add Rental Record to List (Rectangle)
- Process: Calculate Total Rented Books and Rental Summary (Rectangle)
- Process: Display Final Rental Report (Rectangle)
- Output: Show Member Information, Rental Records, and Total Rental Summary (Parallelogram)
- [End] (Oval)





Part 3: Program Code

```
#####
# Community BookHub Rental Tracker
# This program uses OOP to manage book rental records
#####

# -----
# 1. DEFINE THE CLASS
# -----

class Rental:

    # Constructor: Initialize member details
    def __init__(self, member_name, rental_id):

        self.member_name = member_name    # Member name
        self.rental_id = rental_id        # Rental ID
        self.books = []                   # List to store rented books

    # -----
    # METHOD 1: Add Rented Book
    # -----

    def add_book(self, book_name, rental_cost):

        # Add book name and rental cost as a tuple
        self.books.append((book_name, rental_cost))

        print(f"Added book '{book_name}' for {self.member_name}")

    # -----
    # METHOD 2: Calculate Total Cost
    # -----

    def calculate_total_cost(self):

        total = 0

        # Loop through each rented book
        for book in self.books:

            total += book[1]

        return total

    # -----
```

```

# METHOD 3: Display Rental Report
# -----

def display_report(self):

    total_cost = self.calculate_total_cost()

    print("\n" + "-" * 40)
    print(" COMMUNITY BOOKHUB REPORT")
    print("-" * 40)

    print("Member Name:", self.member_name)
    print("Rental ID:", self.rental_id)

    print("\nRented Books:")

    # Display each rented book
    for book in self.books:

        print(f"- {book[0]} : ${book[1]}")

    print("\nTotal Rental Cost: $", total_cost)

    print("-" * 40)

# -----
# 2. MAIN PROGRAM
# -----

# Create rental objects
member1 = Rental("Hillary Star", "RB101")
member2 = Rental("Finn Wolfhard", "RB102")

# Add rented books
member1.add_book("Harry Potter", 15)
member1.add_book("Better Than The Movie", 9)

member2.add_book("Percy Jackson", 12)
member2.add_book("The Alchemist", 7)

# Display final reports
member1.display_report()
member2.display_report()

```

```
# -----  
# END OF PROGRAM  
# Develop by - Eaint Nadi Maung  
# -----
```

- **Results**

COMMUNITY BOOKHUB REPORT

Member Name: Hillary Star

Rental ID: RB101

Rented Books:

- Harry Potter : \$15
- Better Than The Movie : \$9

Total Rental Cost: \$ 24

COMMUNITY BOOKHUB REPORT

Member Name: Finn Wolfhard

Rental ID: RB102

Rented Books:

- Percy Jackson : \$12
- The Alchemist : \$7

Total Rental Cost: \$ 19